OOPS Practical

Q] Write a C++ program for addition of 2 numbers

→

```
#include <iostream.h>
void main()
{
 int a, b, c;

 Cout << "Enter values of a & b";
 Cin >> a >> b;
   C = a+b;
Cout << "Sum =" << c;
}
```

Output:

```
Enter values of a & b
7
2
Sum = 9.
```

Q] Write a C++ program to perform arithmetic operations using switchcase.

→

```
#include <iostream.h>
int main()
{ char operator;
  int a, b, sum, difference, product, quotient, remainder;
Cout << "Enter values of a & b";
Cin >> a >> b;

Cout << "Enter the operator:";
Cin >> Operator;
```

```
switch (opretator) {

case '+':
  sum = a+b;
Cout << "sum: " << sum;
  break;

case '-':
  difference = a-b;
Cout << "difference = " << difference;
 break;

case "*":
product = a* b;
Cout << "product = " << product;
break;

case "/"
quotient = a/b;
Cout << "quotient = " << quotient;
break;

case '%"
remainder = a % b
Cout << "Remainder = " << remainder;
break;
```

```
default:
Cout << "Error"
break;
}
 return 0;
}
```

Output:

```
Enter the operation to be
performed: +
enter values of a & b: 8
2
sum: 10
```

Q] Program to check if the number is even or odd.

→

```
#include <iostream.h>
 int main()
{
 int num;

Cout<<"Enter a number: ";
Cin >> num;

if ((num % 2) == 0)
  Cout << "The number is even";
Else
  Cout << "The number is odd";

return 0;
}
```

Output:

```
Enter a number: 99
The number is odd.
```

Q] Write a C++ code to print 1 to 10 using for loop

→
```
#include <iostream.h>
int main()
{
 int i, n = 10;

for (i=1; i<=n ; i++)
 {
  cout<<i<<"\n";
}
 return 0;
}
```

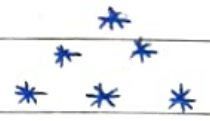
Output: 1
2
3
4
5
6
7
8
9
10

Q] Write a C++ Code to print 10 to 1 using ~~for~~ while loop.

→
```
#include <iostream.h>
int main()
{
 int i=10;
 while (i>=1)
{ cout<<i;
   i--;
  }
 return 0;
}
```

Output: 10
9
8
7
6
5
4
3
2
1

Q] C++ code to print below pattern a]



→

```
#include <iostream.h>
int main()
{
 int i,j;
for(i=1;i<= 3 ;i++) {
   {
    for (j=1;j<=3-i ;j++)
     {
       cout << "  ";
     }
    for(j=1 ; j<=i ;j++)
  cout << " * ";
  }
   cout <<" \n";
   }
    return 0;
}
```

Q] C++ Code to print below pattern

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

→

```
#include <iostream.h>
int main()
{
int i,j;

for (i=1; i<=5; i++) {
for (j=1; j<=i; j++)
{
  cout<<i<<" ";
}
  } cout<<"\n"; }
return 0;
}
```

Q] C++ code to print

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5.
```

→

```
#include <iostream.h>
int main()
{
  int i,j;

  for(i=1; i<=5; i++)
  {
   for(j=1; j<=i ; j++)
  {
   cout<< j<< " ";
  }
   cout<< "\n";
  }
  return 0;
}
```

8/7/25

Experiment -1

Q-

→

```
#include <iostream>
#include <string>
using namespace std;

class Student {
    int roll_no;
    string name;
    string student_class;

public;
  void accept() {
   cout << "Enter Name :";
   cin >> name;
   cout << "Enter Roll no :";
   cin >> roll_no;
  cout << " Enter Class :";
  cin >> student_class;
  }
    void display(){
     cout << "\n --- Student Details --- " << endl;
```

```
    cout<<"Name:"<<name<<endl;
    cout<<"Roll Number:"<<roll_no<<endl;
    cout<<"Class:"<<student_class<<endl;
    }
};
int main();
{
  Student s1;
  cout<<"Enter Student details:"<<endl;
  s1.accept();
  s1.display();
  return 0;
}
```

O/P:

```
Enter Student Details:
Enter Name: Neel
Enter Roll no: 24
Enter Class: SY

--- Student Details ---
Name: Neel
Rollno: 24
Class: SY
```

Q-

```
#include<iostream>
#include<string>
using namespace std;

class Book{
 int b-pages;
double b-price;
string b-name;

public;
 void accept(){
  cout<<"Enter Book Name:";
  cin.ignore();
  getline(cin,b_name);
  cout<<"Enter price of Book:";
  cin>>b_price;
  cout<<"Enter no of pages:";
  cin>>b-pages;
}
 Void display()
  cout<<"\n---Book details---"<<endl
```

```
    cout << "Name of Book : " << b_name << endl;
    cout << "Price of Book : " << b_price << endl;
    cout << "No of pages : " << b_pages << endl;
}
 double getPrice() const{
   return b_price;
 }
};
 int main(){
   Book b1, b2
   b1.accept();
   b2.accept();

  if(b1.getPrice() > b2.getPrice()){
     b1.display();
} else{
    b2.display();
}
 return 0;
}.
```

* O/P.

```
Book Name : It Ends With Us.
Enter Price of Book: 499
Enter No of Pages: 189
Enter Book Name : It start with Us.
```

Enter Price of Book : 498
Enter no of pages: 169.

---: Book Details - - -
Name of Book: It Ends With Us.
Price of Book: 499
No of pages : 189.

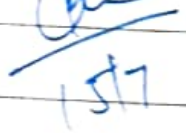
c]

→
```
#include<iostream>
using namespace std;
class Time{
public;
int hours;
int minutes;
int seconds;

public;
void accept(){
cout<<" Enter hours: ";
cin>>hours;
cout<<" Enter minutes: ";
cin>>minutes;
cout<<"Enter seconds: ";
cin>>seconds;
}
int toseconds(){
   return (hours*3600)+(minitcs*60)+seconds;
}
void displayseconds(){
  cout<<"Total seconds:"<<toSeconds()<<endl.
```

```
};
t;
int main(){
  Time t;
t.accept();
t.displayseconds();
return 0;
}
```

O/P

Enter hours: 5
Enter minutes: 10
Enter seconds: 5
Totalseconds: 18605



Experiment-2

a)

→
```
#include <iostream.h>
#include <string.h>
using namespace std;

Class city{
public:
  string name;
  int population

 void getData(){
   cout<<"Enter City Name:";
   cin >> name;
   cout<<" Enter Population :";
   cin>> population;
 }
 void displayData(){
   cout<< "City :"<<name<<" Population :"<< populationcout
 }
};
```

```
int main(){
City cities[5];
int i, maxIndex=0;

for(i=0;i<5;i++){
    cout<<"\n Enter Details of city :";
    cities[i].getData();
}
for(i=1;i<5;i++){
  if(cities[i].population>cities[maxIndex].population
    maxIndex=i;
  }
}

  cout<<"\n City with Highest Population:";
  cities[maxIndex].displayData();
  return 0;
}
```

O/P:

Enter details for city 1:
Enter city Name: Mumbai
Enter population: 60000

" : Pune
" : 50000

" : Banglore
" : 35000

" : Delhi
" : 40000

" : Nashik
" : 25000

City with highest population:
City: Mumbai, Population: 60000

b]

→
```
#include <iostream.h>
using namespace std;

class Account {
public:
   int accNo;
   float balance;

 void accept(){
   cout<<"Enter Account Number:";
   cin>>accNo;
  cout<<" Enter Balance:";
   cin>>balance;
 }
  void giveInterest(){
  if (balance>=5000){
     cout<<"Account No:"<<accNo<<"Updated Balance
      <<balance<<endl;
    }
  }
 };
  int main(){
    Account a[4];
```

```
for (int i=0; i<4; i++){
    cout<<" Account: "<<i+1<<": "<<endl;
    a[i].accept();
}
cout<<"Updated:"<<endl;
For (int i=0 ; i<4 ; i++ ){
    a[i].giveInterest();
    a[i].display();
}
return 0;
}
```

O/P:

Account 1:
Enter account number: 123
Enter Balance: 5000
Account 2:
" : 147
" : 6000
Account 3:
" : 963
" : 2960
Account 4
" : 852
" : 1000

→ Updated:
Account No: 123, Up'Bal: 5500
Account No: 147, Up'Bal: 6600

c]

→
```
#include <iostream.h>
#include <string.h>
using namespace std;

class staff {
public:
  string name;
  string post;

  void accept(){
    cout<< "Enter Staff Name:";
    cin>> name;
    cout<< "Enter post:";
    cin>> post;
  }
  void display(){
    if(post=="HOD" || post=="hod"){
      cout<< name <<" is HOD"<<endl;
    }
  }
};
  int main(){
  staff s[5];
```

```
cin.ignore();

for(int i=0;i<5;i++){
    cout<<"Staff: "<<i+1<<endl;
    s[i].accept();
}
cout<<"list of staff who are HOD:"<<endl;
  for(int i=0;i<5;i++){
     s[i].displayifHOD();
  }
  return 0;
}.
```

→ O/P:

```
Staff 1:
Enter staff name: Sanika
Enter post: Hod
Staff 2:
   "        : Anushka
   "        : Staff
Staff 3:
   "        : Spruha
   "        : Hod
Staff 4:
   "        : Sahil
   "        : Hod
Staff 5:
   "        : Neel
   "        : CEO

list of staff who are HOD:
Sanika is HOD.
Spruha is HOD
Sahil is HOD.
```

12/8.

Experiment-3

a]

→
```
#include<iostream.h>
#include<string.h>
using namespace std;

class Book{
 private:
   string book_title;
   string ~~book~~ author_name;
    float price;

 Public:
    Book(){

   void acceptData(){
     cout<<" Enter Book Title:";
     cin>> book_title;
     cout<<" Enter Author Name:";
     cin>> author_name;
     cout<<" Enter price:";
      cin>> price;
}
```

```
    void displayData(){
      cout << "Book Title : " << book_title << endl;
      cout << "Author Name : " << author_name << endl;
      cout << "Price : " << Price << endl;
    }
};
int main(){
    Book *bookPtr = new Book();
    cout << "Enter book details :" << endl;
    bookPtr -> acceptData();
    cout << "\n Displaying Book details \n" << endl;
    bookPtr -> displayData();

    return 0;
}
```

O/P:

```
Enter Book details:
Enter Book Title: abc
Enter Book Author Name: xyz
Enter price: 499.99
Displaying Book Bio Details:
Book Title: abc
Author Name: xyz
Price: 499.99.
```

b]

→
```
#include <iostream.h>
using namespace std;
class Student {
private:
  int roll-no;
  float percentage;
public:
  Student(){}

  void acceptData(){
    cout<<"Enter Roll Number:";
    cin>>this->roll-no;
    cout<<"Enter Percentage:";
    cin>>this->percentage;
  }
  void displayData(){
    cout<<"Roll Number:"<<this->roll-no<<endl;
    cout<<"percentage:"<<this->percentage<<endl;
  }
};
  int main(){
  Student student;
  cout<<"Enter Student Details:";<<endl;
```

```
    student.acceptData();

    cout<<"Displaying Student Details:\n"<<endl;
    student.displayData();
    return 0;
}
```

O/P:

```
Enter Student Details:
Enter a roll number: 24
Enter percentage: 87.4

Displaying Student Details:
Roll Number: 24
Percentage: 87.4
```

c]

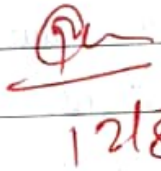
→

```
#include <iostream.h>
using namespace std;

Class student {
Public:
 Class marks{
  Public:
  ~~Private~~:
   int m1, m2, m3;

  void getInput (){
   cout<<"Enter marks for 3 subjects : ";
   cin>> m1 >> m2 >>m3;
  }
  Float getpercentage (){
   int total= m1+m2+m3;
    return total /3.0;
   }
  }
};
 int main()
{
   student :: marks masks;
   maks.getInput ();
```

```
float percentage = marks.getpercentage();
cout<<" Percentage = " << Percentage << "%. "<<endl;
return 0;
}
```

O/P:

Enter Marks for 3 Subjects: 95
98
97
Percentage: 96.66%.



1218

Experiment-4

a]

→
```
#include <iostream>
using namespace std;
Class Number {
private:
  int value;
public:
 void getValue(){
 cout<<"Enter value:";
 cin>>value;
}
void SwapValues(Number &obj) {

   int temp = value;
  value = obj.value;
  obj.value = temp;
  }
};
```

```
int main (){
Number n1, n2;
 cout << " Enter first Number: ";
 n1.input ();
 cout << " Enter second Number: ";
 n2. input();
 n1. swap (n2)
 cout << " After swapping ";
 cout << " n1 = " ; n1. display ();
 cout << " n2 = " ; n2. display ();
 return 0;
}
```

O/P:

Enter First Number:

Enter value: 99

Enter second Number:

Enter value: 100

After swapping:

n1 = 100

n2 = 99

b]

→
```
#include <iostream>
using namespace std;

class Number {
int value;
 Public:
   void input()
{
  cout<<"Enter Value:";
  cin >> value;
}
  void display()
{
  cout << value <<endl;
}
  friend void swap (Number &x, Number &y);
};
void swap (Number & x, Number & y);
{
  int temp = x.value;
  x.value= y.value;
  y.value= temp;
}
```

c]

→

```
int main()
{
 Number n1, n2;
cout << " Enter First Number: ";
 n1.input()
cout << " Enter Second Number: ";
n2.input()
swap(n1, n2);
cout << " After Swapping";
cout << " n1 = "; n1.display()
cout << " n2 = "; n2.display()
return 0;
}
```

c]

→

```
#include<stdio.h>
using namespace std;

class B;

Class A{
  int numA;
public:
  void input(){
```

```
    cout << "Enter number for A:";
    cin >> numA;
}
    void display()
{
    cout << "A: " << numA << endl;
}
friend void swap (A &x, B &y);
};

Class B{
    int numB;
public:
    void input()
{
    cout << "Enter number for B:";
    cin >> numB;
}
    void display()
{
    cout << "B: " << numB << endl;
}
    friend void swap (A &x, B &y);
};
void swap (A &x, B &y)
```

```
{
    int temp = x.numA;
    x.numA = x.numB;
    y.numB = temp;
}
int main()
{
 A obj1;
B obj2;
 obj1.input();
 obj2.input();
swap(obj1, obj2);
cout<<"After swapping :"<<endl;
 obj1.display();
obj2.display();
return 0;
}
```

d]

→

```
#include <iostream>
using namespace std;

Class Result;
Class Result1 {
 private:
 float mark1;
 Public:
 void accept() {
 cout<<" Enter Marks for first student";
 cin>> mark1;
 }
friend float Average(Result1, Result2);
};
Class Result2 {
 Private:
 float Mark2;
 Public:
 void accept() {
 cout<<" Enter Marks for student 2:";
 cin>> Marks2;
};
```

```
friend float Average (Result1 , Result2)
};
Float Average (Result1 a , Result2 b);
    Float average= (a.mark1 + b.mark2);
                   2;
    return average;
}
 int main (){
  Result 1 r1;
  Result 2 r2;
  Float avg;

  r1.accept();
  r2.accept();
  avg= Average (r1, r2);
  cout<<" Average of 2 numbers: "<< average<<
  endl;
 return 0;
}
```

→ O/P:

```
Enter Marks for Student 1: 70
Enter Marks for Student 2: 60
Average of 2 Result is 65.
```

c]

→ #include <~~stdio.h~~ iostream.h>
using name space std;

Class Number2;
Class Number1 {
private:
int num1;
Public:
~~in~~ void accept(){
cout<< "Enter Number:";
cin >> num1;
}
Friend void findGreatest (Number1, Number2);
};

Class Number2{
private:
int num2;
Public:
void accept(){
cout<<" Enter Number:";
cin>>num2;
}
Friend void find Greatest (Number1, Number2

```
};
void Find Greatest (Number1.a, Number2.b);
 {
   if (a.num1 > b.num2){
   cout<<" Greatest Number is: "<<a.num1<<endl;
  else if (b.num2 > num1){
   cout<< Greatest Number is: "<<b.num2<<endl;
  } else {
    cout<<" Both are equal: "<<endl;
  }
}
 int main{
 Number1. obj1;
 Number2. obj2;
 Find greatest (obj1, obj2),
 return 0;
 }.
```

→ O/P:

```
Enter Number: 8
Enter Number: 7
Greatest Number is 8.
```

f]

→
```
#include <iostream.h>
using namespace std;

class classB;
class classA{
private:
  int valueA;
Public:
  void input(){
  cout<<" Enter value of classA:";
    cin>>valueA;
  }
  friend int sum(classA a, classB b);
};
class classB{
private:
   int valueB;
Public:
  void input(){
    cout<<" Enter value of classB:";
    cin>>valueB;
  }
```

```
int main()
  classA objA;
  classB objB;

  objA.input()
  objB.inputO:
 cout<< "Sum: "<< sum (objA,objB) <<endl;
 return 0;
}.
```

g]
```
# include <iostream>
using namespace std;
class cube;

class box {
private:
    int volume;
public:
void accept()
{
  cout<<" Enter the volume of Box: ";
  cin>> volume;
}
Friend void findgreater (box, cube);
};
```

```
class cube{
private:
   int volume;
public:
 void accept()
 {
  cout<<"Enter the Volume of cube:";
  cin>> volume;
 }
void gfind greater (box b, cube c)
  {
   if (b.volume >c.volume)
     {
       cout<<" Volume of box is greater.";
     }
     else if (b.volume <c.volume)
     {
       cout<<"Volume of cube is greater";
     }
     else
     {
       cout<<" both volume are equal.";
     }
}
int main(){
```

```
Box h;
cube g;
h.accept();
g.accept();
Find greator(h,g);
return 0;
}.
```

9]

→
```
#include<iostream.h>
using namespace std;
class Complex{
private:
  float real;
  float imag;
public:
 void accept(){
   cout<<"Enter Real part:";
   cin>>real;
   cout<<"Enter imaginary part:";
   cin>>imag;
}
 void display(){
   cout<<real<<"+"<<imag<<"i"<<endl;
}
```

```
friend Complex add Complex (Complex c1, Complex c2);
};
Complex add Complex (Complex c1, Complex c2) {
    Complex temp;
    temp.real = c1.real + c2.real;
    temp.imag = c1.imag + c2.imag;
    return temp;
}
int main(){
    Complex num1, num2, sum;

    cout<<" Enter first Complex Number :"<<endl;
    num1.accept();

    cout<<" Enter Second Complex Number:"<<endl;
    num2.accept();

    sum = add Complex (num1, num2);
    cout<<" Sum of Complex Number : ";
    sum.display();
    return 0;
}
```

11]

→
```
#include<iostream>
using namespace std;
class student{
private:
  string name;
  Float mark1, mark2, mark3;
Public:
  void accept(){
    cout<<"Enter Student Name:";
    cin>>name;
    cout<<" Enter Student Marks in 3 subject:";
    cin>>mark1>>mark2>>mark3;
  }
  Friend void calculate Average (Student s);
};
  void calculate Average (Student s){
    Float avg = (s.mark1+s.mark2+s.mark3)/3;
  cout<<"\n Student Name:"<<sname<<endl;
  cout<<" Average Marks:"<<avg<<endl;
  }
  int main(){
  Student stu;
  stu accept()
  calculate Average (stu);
  return 0;
}
```

12]

```
#include <iostream>
#include <cmath>
using namespace std;

class Point{
private:
  float x;
  float y;
Public:
  void accept(){
    cout << "Enter coordinate (x,y): ";
    cin >> x >> y;
  }
  Friend float calcDistance(Point, Point);
};
float calcDistance(Point p1, Point p2){
  float dx = p2.x - p1.x;
  float dy = p2.y - p1.y;
  return sqrt(dx*dx + dy*dy);
}
int main()
  Point A,B;
  A.accept();
  B.accept();
  float distance = calcdistance(A,B);
  cout << "The distance betn the 2 point is" << distance;
}
```

Q13]

```
#include <iostream.h>
using namespace std;
class Beta;
class Gamma;
class Alpha{
private:
  int a;
 public:
  void accept(){
   cout<<"Enter value for Apha:";
   cin>>a;
 }
 friend void sumValues(Alpha,Beta,Gamma)
};
 class Beta;
 private:
  int b;
 public
  void accept(){
     cout<<"Enter Value for Beta:";
     cin>>b;
 }
  friend void sum Values(Appha, Beta, Gamma);
};
```

```
class Gamma{
private:
  int c;
public:
 void accept(){
    cout<<" Enter value for Gamma:";
    cin>>c;
 }
 friend void sumValues(Alpha, Beta, Gamma)
};
void sumValues (Alpha x, Beta y, Gamma z){
   int sum = x.a+y.b+z.c;
   cout<<" Sum of all values = "<<sum<<endl;
 }
int main(){
  Alpha objA;
  Beta objB;
  Gamma objC;

  objA.accept ();
  objB.accept();
  objC.appet();

  Sum Value(objA, objB), objC)
 return 0;
}
```

14/8

Experiment - 5

Q1] Copy Constructor:

Output:
Sum From 1 to 10 is: 55

```
#include <iostream>
using namespace std;

class SumCalculator {
private:
    int n, sum;
public:
    SumCalculator(int num) : n(num), sum(0){
        for(int i=1; i<=n; ++i)
            sum += i;
    }
    void display(){
    cout<< "Sum From 1 to " << n << "is:" << sum << endl; }
};
int main(){
    SumCalculator calc(10);
    calc.display();
    return 0;
}
```

- Using Default Constructor :

```
#include <iostream>
using namespace std;
class Sum{
  private:
    int n;
  public:
    Sum(){
      n=10;
      cout<<"n is set to: "<<n<<endl;
    }
    int calculateSum(){
      int sum=0;
      for(int i=1;i<=n;i++){
        sum+=i;
      }
      return 0;
    }
};
int main(){
    Sum sum1;
    cout<<"Sum of numbers from 1 to "<<10<<" is: "
    <<sum1.calculateSum()<<endl;
    return 0;
}
```

Output:
n is set to 10
Sum of numbers from 1 to 10 is: 55

Using Parameterized Constructor:

→
```
#include<iostream>
Using namespace std;
class Sum{
private:
    int n;
public:
    Sum(int x){
        n=x;
        cout<<" n is set to "<<n<<endl;
    }
int calculateSum(){
    int sum=0;
    for(int i=1;i<=n;i++){
        sum+=i;
    }
    return sum;
}
int main(){
    int n;
cout<<" Enter a value for n: ";
cin>>n;
Sum sum2(n);
cout<< Sum of number Foram 1 to "<<n<<is :>>sum2.
calculate Sum()<<endl;
return 0;}.
```

Output:

Enter a value for n 10

n is set to 10

Sum of numbers from 1 to 10 is: 55

Q2]

Using Parameterized Constructor:

→

```
#include <iostream>
#include <string>
using namespace std;
private:
   string name;
   float percentage;
public:
   Student (string n, float p){
        name=n;
        percentage =p;
}
 voiddisplay(){
    cout<< "Student Name : "<<name <<endl;
    cout<< "Percentage : "<< percentage<<"%. "<<endl;
}
};
int main(){
   string studentName;
   float studentPercentage;
   cout<< "Enter student's name: ";
   getline(cin, studentName);
```

```
cout<<" Enter student's Percentage: ";
cin>> studentPercentage;
student student1(studentName, studentPercentage);
student1.display();
return 0;
}
```

→ Output:

```
Enter student's Name: Neel Sonawane
Enter student's Percentage: 85%
Student Name: Neel Sonawane.
Percentage: 85%.
```

- Using Copy Constructor:

→

```
#include<iostream>
#include<string>
using namespace std;

class student{
private:
    string name;
    float percentage;
public:
    student(){
        name = "Unknown";
```

```
    percentage = 0.0;
}
Student (const Student &s) {
    name = s.name;
    percentage = s.percentage;
}
void display() {
    cout << "Student Name: " << name << endl;
    cout << " Percentage: " << percentage << endl;
}
};
int main() {
    Student student1;
    Student student2(student1);
    cout << " student 1:\n";
    student1.display();
    cout << " student 2:\n ",
    student2.display()
    return 0;
}
```

→ Output:

```
Student Name: Neel.
Percentage: 90%.
Student Name: Neel
Percentage: 90%.
```

Using Default Constructor:

→
```
#include <iostream>
#include <string>
using namespace std;
class Student{
private:
    string name;
    float percentage;
public:
    student(){
        name = "Neel Sonawane";
        percentage = "97%.";
    }
void display(){
    cout<< "Student Name : " << name << endl;
    cout<< "Percentage: " << percentage << endl;
    }
};
int main(){
    student student1;
    cout<< Student1 \n";
    Student1.display();
    return 0;}.
```

→ Output:

```
Student Name: Neel Sonawane.
Percentage: 97%.
```

Q3

```
#include <iostream>
#include <string>
using name space std;
class College{
private:
    int roll_no;
    string name;
    string course;
public:
    College(int r ,string n,string c="CSE"){
        roll_no=r ;
        name =n.
        course=c;
    }
void display(){
    cout<<"Roll no:"<<roll_no<<endl;
    cout<<"Name:"<<name<<endl;
    cout<<"Course:"<<course<<endl;
  }
};
 int main(){
    int roll_no1,roll_no2;
    string name1,name2,cours1, course2;
```

```
cout<<"\n Enter Roll No, Name & Course for Student 1:"
cout<<"Roll no:";
cin>>roll_no1;
cin.ignore();
cout<<"Name:";
getline(cin, name1);
cout<<"Course (Press Enter for Default):";
getline(cin, course1);
cout<<"\n Enter Roll no, Name & Course for student 2:"<<endl
cout<<"Roll no:";
cin>>roll_no2;
cin.ignore();
cout<<"Name:";
getline(cin, name2);
cout<<"\n Course (Press Enter For Default):";

College student1(roll_no1, name1, course1.empty() ?
"CSE" : course1);
College student2(roll_no2, name2, course2.empty() ?
"CSE" : course2);
cout<<"\n student 1 Data:"<<endl;
student1.display();
cout<<"\n student 2 Data:"<<endl;
student2.display();
cout<<"\n return 0;
}
```

→ Output:

Enter Roll no', Name & Course for first student:
Roll no': 25
Name: Neel Sonawane
Course (Press Enter for Default): AI DS

Enter Roll no', Name & Course for Second student:
Roll no': 28
Name: Salil Nilawar
Course (Press Enter for Default):

Student 1 data:
Roll no: 25
Name: Neel Sonawane
Course: AI DS

Student 2 data:
Roll no: 28
Name: Salil Nilawar
Course: CSE

Experiment-6

Q1] Single Inheritance.

→
```
#include <iostream>
using namespace std;
class Person {
protected:
    string name;
    int age;
public:
    void setPersonDetails(string n, int a) {
        name = n;
        age = a;
    }
    void displayPersonDetails() {
        cout << "Name : " << name << endl;
        cout << "Age: " << age << endl;
    }
};
class student : public Person{
  private:
    int rollNumber;
  public:
    void setStudentDetails(string n, int a, int r) {
```

Output:

```
Enter Name: Neel
Enter Age: 16
Enter Roll no: 24
Student Details:
Name: Neel
Age: 16
Roll no: 24.
```

```
    setPersonDetails(n,a);
    rollNumber=r;
  }
  void displayStudentDetails(){
    displayPersonDetails();
    cout<<"Roll Number:"<<rollNumber<<endl;
  }
};
int main(){
  Student s1;
  string Name;
  int age,roll;
  cout<<"Enter Name:";
  getline(cin,name);
  cout<<"Enter Age:";
  cin>>age;
  cout<<"Enter Roll Number:";
  cin>>roll;
  s1.SetStudentDetails(name,age,roll);
  cout<<"\n---Student Details---"<<endl;
  s1.displayStudentDetails();
  return 0;
}
```

Q2] Multiple Inheritance

→
```
#include <iostream>
using namespace std;
class Academic{
protected:
    int marks;
public:
  void setMarks(int m){
      marks=m;
    }
  void displayMarks(){
     cout<<"Academic Marks:"<<marks<<endl;
    }
};
class Sports{
protected:
  int sportsScore;
public:
  void setsportsScore(int s){
     sportsScore=s;
  }
  void displaySportsScore(){
     cout<<"Sports Score:"<<sportsScore<<endl;
   }
};
```

```
class Result : public Academic , public Sports{
private:
  int total;
public:
  void calculateTotal() {
    total = marks + sportsScore;
  }
  void displayResult() {
    displayMarks();
    displaySportsScore();
    cout<< "Total Score:" <<total <<endl;
  }
};
int main() {
  Result r;
  int m, s;
  cout<<"Enter Academic Marks:";
  cin>>m;
  cout<<"Enter Sports Score:";
  cin>>s;
  r.setMarks(m)
  r.setSportsScore(s);
  r.calculateTotal();
  cout<<"\n--- Student Result ---"<<endl;
  r.displayResult();
  return 0;
}
```

Q3] Multiple Inheritance:

→
```
#include <iostream>
using namespace std;
class Vehicle {
protected:
    string brand;
    string model;
public:
    void setVehicleDetails(string b, string m){
        brand = b;
        model = m;
    }
    void displayVehicleDetails(){
        cout<<"Brand:" <<brand<<endl;
        cout<<" Model:" << model <<endl;
    }
};
class Car : public Vehicle{
protected:
    string type;
public:
    void setCarDetails(string b, string m, string t){
        setVehicleDetails(b, m);
        type=t;
    }
```

```
void displayCarDetails(){
    displayVehicleDetails();
    cout<<"type : "<<type<<endl;
  }
};
class ElectricCar : public Car{
private:
   int batteryCapacity;
public:
   void setElectricCarDetails(string b,string m,string t,int bc){
      setCarDetails(b,m,t);
      batteryCapacity=bc;
   }
   void displayElectricCarDetails(){
      displayCarDetails();
      cout<<"Battery Capacity: "<<batteryCapacity<<endl;
   }
};
int main(){
   ElectricCar c1;
   string brand, model, type;
   int battery;
   cout<<"Enter Vehicle Brand: ";
   getline(cin,brand);
```

```
cout<<"Enter Vehicle Model: ";
getline(cin,model)
cout<<"Enter Car type (eg - Seden, SUV);
getline(cin, type)
cout<<"Enter Battery Capacity (kWh): ";
cin>>battery;
el.setElectricCarDetails(brand, model, type, battery);
cout<<"\n--- Electric Carr Details ---";
el.displayElectricCarDetails();
return 0;
}
```

Q1] ~~Heich~~ Heirarchical Inheritance

→
```
#include<iostream>
using namespace std;
class Employee{
protected:
  int empID;
  string name;
public;
  void setEmployeeDetails(int id, string n){
    empID = id;
    name = n;
  }
  void displayEmployeeDetails(){
    cout<<" Employee ID : "<<empID<<endl;
    cout<<" Name : "<<name<<endl;
  }
};
class Manager : public Employee{
private:
  string department;
public:
  void setManagerDetails(int id, string n, string d){
    setDetails(id, n);
    department = d;
  }
```

```
void displayManagerDetails(){
    displayEmployeeDetails();
    cout<<"Department:"<<department<<endl;
  }
};
class Developer : public Employee{
private:
  string programminglang;
public:
  void setDeveloperDetails(int id, string n, string pl){
      setEmployeeDetails(id, n);
      programming lang = pl;
  }
  void display ~~Development~~ DeveloperDetails(){
      displayEmployeeDetails();
      cout<<"Programming language:"<<programminglang;
  }
};
int main();
  Manager m;
 developer d;
int id;
 cout<<"EnterManagerId:"
 cin>>id;
```

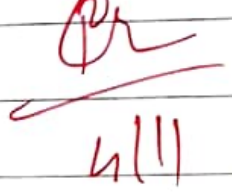
```
cout << " Enter Manager Name: ";
getline (cin, name);
cout << " Enter Department : ";
getline (cin, dept);
m.setManagerDetails (id, name, dept);
cout << " \n Developer ID: ";
cin >> id.
cout << " Enter Developer Name: ";
getline (cin, name);
cout << "Enter Programming language: ";
getline (cin, lang);

d.setDeveloperDetails(id, name, lang);
cout << " \n Manager Details " << endl;
m.displayManagerDetails();
cout << " \n Developer Details " << endl;
d.displayDeveloperDetails();
return 0;
}
```

Q6]

```
#include <iostream>
using namespace std;
class college :student
{
 protected:
    int student_id;
    int college_code;
public:
    void input student()
   {
     cout<<"Enter student id:";
     cin>> student-id;
     cout<<"Enter college code:";
     cin>> college-code;
   }
   void display student()
   {. cout<<"student ID:"<< student-id <<endl;
    cout<<"college code:"<<college-code<<endl;
   }};
class test: virtual public college student {
  protected:
     float percentage;
  public:
     void Input Test()
     {.
```

```
    cout<<"Enter Test Percentage:";
    cin>>percentage;
  }
void displayTest()
{
  cout<<"Test Percentage:"<<percentage<<"%."<<
};
class sports: virtual public college_student
protected:
    char grade;
Public:
   void input sports()
  {
  cout<<"Enter sports Grade:";
  cin>>grade;
};
  void display sports (){
 cout<<"sports Grade:"<<grade<<endl;
};
 public:
    void Input Result(){
    input student();
    input test();
    input sports();
};
```

```
void displayResult()
{
  displaystudents;
  displaytest();
  displaysports();
};
int main(){
   Result r;
   r.inputResult();
   r.displayResult();
   return 0;
};
```



Experiment-7

a]

→
```
#include<iostream>
using namespace std;
class area{
  public:
   int calculate(int l, int b)
  {
    return l*b;
  }
   int calculate(int side)
 {
  return side*side;
}};
 int main() {
 area a;
 cout<<"Area of laboratory (Rectangle 10x5): "
    << a.calculate(10, 5)<<endl;
 cout<<"Area of classroom (square 6x6): "<<
    a.calculate(6)<<endl;
   return 0;
 }
```

b]

→
```
#include <iostream>
using namespace std;
class sum {
public:
float add (float a, float b, float c, float d, float e) {
  return (a+b+c+d+e);
}
int add (int a1, int a2, int a3, int a4, int a5, int a6, int a7,
int a8, int a9, int a10) {
  return (a1+a2+a3+a4+a5+a6+a7+a8+a9+a10);}
};
int main() {
  sum s;
cout<<"sum of 5 float : "<<s.add (1.1f, 2.2f, 3.3f, 4.4f, 5.5f,
6.6f, 7.7f, 8.8f, 9.9f
cout<<"sum of 10 integers : "<< s.add(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
  <<endl;
return 0;
}
```

c] #include <iostream>
using namespace std;
class Numbers {
int x;
public:
Number (int a) { x=a; }
void display() {
cout<<" Value : " << x <<endl;
}
void operator -()
{
x= - x;
}};
int main() {
Number n(10);
n.display ();
-D;
n.display ();
return 0;
}

d]

```
#include <iostream>
using namespace std;
class Number
{
 int x;
Public:
  Number(int a) {
  x=a;
}
 void display()
{
 cout<<"value:"<<x<<endl; }
void operator ++() {
  ++x;
}};
 int main (){
 number n(5);
 n display();
 ++n;
 n display();
 n++;
 n display();
  return 0;
}
```

5
6
7

Experiment 8

a]

→
```
#include <iostream>
#include <string>
using namespace std;
class addstring {
  string str;
Public:
  Addstring (string s)
{
 str=s;
}
addstring operator + (addstring obj ) {
    return addstring (str + obj.str);
}
};
int main () {
 addstring s1("XYZ");
 addstring s2("PQR");
 addstring s3 = s1 + s2;
 cout<<" concentrated string :"<< s3.str <<endl;
 return 0;
}.
```

Q2]

```
#include <iostream.h>
#include <string.h>
using namespace std;
class I-login {
 protected:
   string name;
 public:
  virtual void accept (){
    cout <<"\n Enter Name : ";
    cin >> name;
   }
  virtual void display () {
    cout<<"\n Name : "<<name <<endl;
   }
};
class email-login : public I-login {
protected:
  string email;
  string password;
public:
  void accept() overide {
   I-login :: accept ();
   cout <<" Enter Email : ";
   cin >> email;
```

```
cout<<"Enter Password:";
cin>>Password;
}
void display() override {
I-login::display();
cout<<"Email:"<<email<<endl;
cout<<"Password:"<<password<<endl;
}
};
class membership-login:public email-login{
    int member-id;
public:
void accept() override{
email-login::accept();
cout<<"Enter Member ID:";
cin>>member-id;
}
void display() override{
cout<<"\nMembership login Details:"<<endl;
email-login::display();
cout<<"Member-ID:"<<member-id<<endl;
}
};
```

```
int main() {
  membership_login m;
  m.accept();
  m.display();
  I_login* login = new membership_login();
  login->accept();
  login->display();
  deletelogin;
  email_login e;
  e.accept();
  e.display();
  return 0;
}
```

Experiment - 9.

Q1]

→

```
#include <iostream>
#include <fstream>
using namespace std;
int main() {
  fstream new_file;
  new_file.open("new_file.txt", ios::out);
  if (!new_file){
    cout <<" file creation failed! "<<endl;
  }else{
    cout <<" New file created"<< endl;
    new_file.close();
  }
  return 0;
}
```

22]

```
#include <iostream.h>
#include <fstream>
using namespace std;
int main(){
 ofstream File1("File1.txt",ios::app);
 if (!File1){
  cout<<"Error oppening File1.txt "<<endl;
 return 1;
 }
 File1 <<"Welcome to MIT" <<endl;
 File1.close();
 ifstream File1_read("File1.txt");
 ofstream File2("File2.txt");
 if(!File1_read ){
  cout <<" Error openin g File1.txt for reading "<<endl;
  return 1; }
 if(!File2){
  cout<<" Error opening File2.txt for writing"<<endl;
  return 1;}
 char ch;
 while (File1_read.get (ch)) {
   File2.put (ch);}
 cout <<"Content copied success fully From File1.txt to File2.txt"<<en
 File1_read.close();
 File2.close ();
```

Q3]

```
#include <iostream>
#include <fstream>
using namespace std;
int main(){
 ifstream inputFile ("input.txt");
 if(!inputFile)
 {
   cout<< "cannot open input.txt!" <<endl;
   return 1;
 }
char ch;
int spacecount=0;
int digitcount=0;
while (inputfile.get(ch))
{
  if (ch==' ')
     space count++;
  else if (isdigit(ch))
     digit count++;
}
 inputFile.close();
cout<< "Total spaces:" <<spacecount<<endl;
cout<< "Total digit:" <<digitcount<<endl;
```

Experiment-10

Q1]

```
#include <iostream>
using namespace std;
template <class T>
T myfunction (T a[] ,int n){
 T sum=0;
 for(int i=0;i<n;i++){
    sum =sum+a[i];
 }
 return sum;
}
int main(){
  int a1[3] = { 2, 0, 2};
float a2[3] = { 1.2, 1.3, 2.4};
double a3[4]={10.5, 20.5, 30.5};

cout<<"sum of integer array is :"<<myfunctional(a1,3)
<<endl;
cout<<" sum of float array is: "<<myfunction(a2, 3)<<endl;
cout<<" sum of double array is: "<<myfunction (a3,3)<<endl;
 return 0;
}
```

```
#include <iostream>
using namespace std;
template <class T>
T square (T num) {
   return num * num;
}

  template <>
  string square <string>(string str) {
      return str + str;
    }
  int main(){
    int i = 5;
    string s = "Hello";
    cout<< "square of integer" << i << " = " << square(i) << endl;
    cout<< "square of string" << s << " \ " << square(s) << endl;
   return 0;
  }
```

Q3]

→
```
#include <iostream>
using namespace std;
template <class T>
class Calculator {
 T a, b;
public:
  Calculator (T x, T y) {
    a = x;
    b = y;
  }
T add() { return a+b; }
T sub() { return a-b; }
T prod() { return a*b; }
T div() { if (b != 0)
    iB   return a/b;
        else{
          cout << "Error! " << endl;
          return 0;
        }
    }};
  int main() {
calculator <int> c1(10, 5);
cout << "Addition = " << c1.add() << endl;
cout << "Subtraction = " << c1.sub() << endl;
cout << "Multiplication = " << c1.prod() << endl;
```

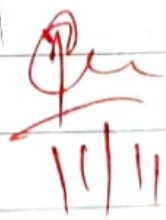
```
cout << " Division = " << c1.div() << endl;
return 0;
}
```

Q4]

→

```
#include <iostream>
using namespace std;
template <class T>
class stack{
 T stk[10];
 int top;
 public:
 stack() {top= -1;}
 void push(T val) {
   if (top==9)
    cout<< 'Stack Overflow! "<<endl;
  else
    stk[++top]=val;
 }
 void pop(){
  if (top == -1)
    cout<<" stack underflow! " << endl;
  else
   cout<<" Popped: " << stk[top--] <<endl;
  {
```

```
void display (){
 if (top == -1)
   cout << "stack is empty!" << endl;
 else{
   cout << " Stack : ";
   for (int i = top; i >= 0; i--)
      cout << stk[i] << " ";
   cout << endl;
 }
}
};
int main(){
 Stack <int> s;
 s.push(10);
 s.push(20);
 s.push(30);
 s.display();
 s.pop();
 s.display();
 return 0;
}
```



Experiment-11

Q1]

→

```
#include <iostream>
#include <vector>
using namespace std;
int main(){
    vector<int> v={1,2,3,4,5,6,7,8,9,10};
    cout<<"Initial Vector:"<<endl;
    for(int i=0; i<10; i++){
        cout<<v[i]<<" "<<endl;
    }
    cout<<"Multiply by 10"<<endl;
    for(int i=0;i<10;i++){
        v[i]=v[i]*10;
    }
    cout<<"New Vector:"<<endl;
    for(int i=0; i<10; i++){
        cout<<v[i]<<" "<<endl;
    }
    return 0;
}
```

Q2]

→
```
#include <iostream>
#include<vector>
using namespace std;
int main() {
  vector<int> v = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
  cout << "Initial Vector: " <<endl;
  for(vector <int> ::iterator it=v.begin();
  it != v.end(); ++it)
  {  cout<< *it << "  " << endl;
  }
  int n;
  cout<< "Enter Scaler Value: "<<endl;
  cin >>n;
   for(vector <int>::iterator it = v.begin(); it!=
   != v.end(); ++it) {
   *it = (*it)*n;
   }
   cout << *it << "  " <<endl;
    for(vector <int> ::iterator it = v.begin();
    it != v.end(); ++it)
     {
       cout<< *it<< "  " <<end;
     }
  return 0;
}
```

New Vector"

Experiment 12.

Q1]

```
#include <iostream>
#include <stack>
using namespace std;
int main() {
  stack<int> marks;
  int choice, value;
  cout<<" menu: \n";
  cout<<" 1. Push: \n";
  cout<<"2. Pop: \n";
  cout<<"3. Display \n";
  cout<<"4. Exits \n";
  do {
  cout<<"\n Enter your choice: ";
  cin>>choice;
  switch(choice){
   case 1:
     cout<<" Enter value to push: ";
     cin>>value;
     marks.push(value);
     break;
```

```
case 2:
 if (marks.empty()){
   cout<<"Stack is empty\n";
  }else{
    cout<<"Popped : "<<marks.top()<<endl;
    marks.pop();
  }
case 3:
 if (marks.empty ()) {
   cout<<"Stack is Empty.";
 } else{
   cout<<"Stack elements (top to Bottom): ";
   stack<int> temp =marks;
   while (!temp.empty()) {
      cout <<temp.top()<<" ";
      temp.pop();
    }
    cout<<endl;
  }
  break;
 case 4:
    exit(0);
 default:
    cout<<"Invalid choice\n";
  }
} while (choice!=4)
```

Q2]

→

```
#include <iostream>
#include <queue>
using namespace std;
int main() {
  queue <int> marks;
  int choice, value;
  cout<<"Menu:\n";
  cout<<"1. Enqueue:\n";
  cout<<"2. Dequeue:\n";
  cout<<"3. Display:\n";
  cout<<"4. Exit\n";
  do {
    cout<<" Enter your choice:";
    cin>>choice;
    switch (choice) {
    case 1:
      cout<<"Enter value to enqueue:";
      cin>>value;
      marks.push(value);
      break;
    case 2:
      if (marks.empty()) {
        cout<<"Queue is empty\n";
      }else{ cout<<"Dequeued:"<<mark.Front()<<endl
          marks.pop();}
```

```
case 3:
 if (marks.empty()){
   cout<<"Queue is empty. ";
 }else{
   cout<<"Queue elements (front to rear):";
   queue<int> temp = marks;
   while (!temp.empty()){
     cout<<temp.front()<<" ";
     temp.pop();
   }
   cout<<endl;
 }
 break;
case 4:
  exit(0);
default:
   cout<<"Invalid choice\n";
 }
}while (choice != 4)
return 0;
}
```